LeMiCi Blog Platform — End-to-End System Guide

> ***Audit Status: ✅ Verified against actual codebase***

1. System Architecture Overview


page.bpmn
blog-detail-page.bpmn
Assigns Job
Assigns Job
blog-postgres
worker (Go)
Handles:
CREATE_BLOG
UPDATE_BLOG
CREATE_SUBSCRIBER
UPDATE_SUBSCRIBER
CREATE_FOLLOWER
DELETE_FOLLOWER
query-postgresql worker
(internal/workers/data-access/query-postgresql)
Handles BPMN queryType:
BLOG_FEATURED
BLOG_POPULAR
BLOG_LISTING
BLOG_DETAIL
Writes to PG
Queries PG
POSTGRESQL
Tables:
- listings
- blogs
- author_profiles
- listing_stats
POSTGRESQL
(Source of Truth)

```
│  - listing_cats  │
│  - blog_subscribers│
│  - blog_author_  │
│    followers     │
└──────────────────┘
          │
   Cron Sync Script
   (sync-postgres-to-es.go)
          ▼
┌──────────────────────┐
│  ELASTICSEARCH       │
│  Index: blog_listings│
│  (For fast searching)│
└──────────────────────┘
```

Key Architecture Rule:

`/featured` & `/popular` → Skip Zeebe. API Gateway queries Postgres **directly** via `blog_handler.go`. Fast, lightweight.

`/home`, `/listing`, `/:id` → Go through Zeebe BPMN → `query-postgresql` worker → Postgres. More orchestrated.

All writes (`CREATE_BLOG`, `FOLLOW`, `SUBSCRIBE`) → Always via Zeebe → `blog-postgres` worker.

---

2. Database Schema — Every Table Explained

Core: `listings` (Central Registry)

Every blog is a "listing" with `entity_type = 'blog'`.

```
id           UUID PK
name         VARCHAR  -- Blog title
slug         VARCHAR  -- URL: /blog/how-to-start-ai-startup
description  TEXT     -- Rich HTML content
status       VARCHAR  -- DRAFT | PENDING | LIVE | ARCHIVED
entity_type  VARCHAR  -- 'blog' (fixed value)
is_featured  BOOL     -- Shown in featured sections?
created_by   UUID     -- FK → users.id (author)
created_at   TIMESTAMP
```

Extension: `blogs` (Blog-specific metadata)

```
id               UUID  FK → listings.id
reading_time_mins INT   -- "5 min read"
seo_title        TEXT  -- <meta title>
seo_description  TEXT  -- <meta description>
```

```
featured_image_url TEXT  -- S3 URL for hero image
author_display_name VARCHAR -- Pen name shown on post
tags              TEXT[]   -- ['AI', 'Startups', 'Tech']
additional_media_urls TEXT[] -- Array of S3 URLs for attached
PDFs, extra images, etc.
```

New: **`author_profiles`** (Author personalization)

```
user_id             UUID   PK FK → users.id
full_name           VARCHAR  -- Jane Doe
author_name         VARCHAR  -- JaneTech (pen name)
bio                 TEXT     -- "I write about AI and
startups..."
profile_picture_url TEXT     -- S3 URL for avatar
categories          TEXT[]   -- ['Tech', 'Business']
updated_at          TIMESTAMP
```

Engagement: **`listing_stats`**

```
listing_id   UUID  FK → listings.id
view_count   INT   -- Incremented on every blog detail view
(async goroutine)
follow_count INT
```

Categorization: **`listing_categories`**

```
listing_id  UUID FK → listings.id
category_id UUID FK → categories.id
```

Social: **`blog_subscribers`**

```
email           VARCHAR UNIQUE
status          VARCHAR  -- ACTIVE | UNSUBSCRIBED
source          VARCHAR  -- where they subscribed from
unsubscribed_at TIMESTAMP
```

Social: **`blog_author_followers`**

```
follower_id UUID FK → users.id
author_id   UUID FK → users.id
-- UNIQUE(follower_id, author_id) prevents duplicates
```

3. Step-by-Step: Creating a Blog (with Jane Doe Example)

Scenario: User Jane Doe clicks "Create Blog".

Step 1: Author Profile Modal Opens (B4 Blog Creation)

Jane fills:

Full Name: "Jane Doe"

Pen Name: "JaneTech"

Bio: "I write about AI and startups"

Profile picture → uploaded separately via S3

S3 Upload flow:

```
Frontend → GET /api/v1/blog/media/presigned
         ← API Gateway returns: { url:
"https://s3.aws.../presign...", key: "blog/media/uuid.jpg" }
Frontend → PUT https://s3.aws.../presign... (direct browser-to-S3
upload)
         ← S3 returns 200 OK
Frontend → stores final URL locally: "https://s3.../jane-
avatar.jpg"
```

Step 2: Blog Editor Opens
Jane writes content. Uploads Featured Image (same S3 presigned flow). Sets:

Title: "The Future of AI in 2025"

SEO Title, Meta Description

Tags: `["AI", "Tech"]`

Categories: Selected from dropdown (UUID of "Technology" category)

Reading time: 7 mins

Clicks **Publish**

Step 3: Frontend submits `POST /api/v1/blog`

```
{
  "name": "The Future of AI in 2025",
  "slug": "future-of-ai-2025",
  "description": "<p>AI is transforming...</p>",
  "status": "LIVE",
  "reading_time_mins": 7,
  "seo_title": "Future of AI 2025 | LeMiCi",
  "featured_image_url": "https://s3.../blog-hero.jpg",
  "author_display_name": "JaneTech",
  "tags": ["AI", "Tech"],
  "category_ids": ["uuid-of-technology"],

  "author_full_name": "Jane Doe",
  "author_bio": "I write about AI and startups",
```

```
  "author_profile_pic": "https://s3.../jane-avatar.jpg",
  "author_categories": ["Tech", "Business"],

  "additional_media_urls": [
    "https://s3.../extra-chart.png",
    "https://s3.../whitepaper.pdf"
  ]
}
```

Step 4: Save (Draft) vs Publish (Pending Review)
The UI provides two main actions for a blog:

Save as Draft: The frontend sends the payload with `"status": "DRAFT"`. It goes through Zeebe and saves to Postgres. It will not show up in any public API.

Publish: The frontend submits the payload. **Note:** The backend strictly enforces the status as `PENDING_REVIEW` to ensure quality control. The blog is NOT immediately visible on the platform.

Step 5: API Gateway (`blog_handler.go: CreateBlog`)

Extracts `userId` from JWT session → injects `"created_by": "jane-uuid"`

Adds `"operation_type": "CREATE_BLOG"`

Injects `"operationsEmail"` from API Gateway config.

Serializes entire JSON as string → puts in `variables["payload"]`

Generates `correlationKey` UUID for response matching via Redis PubSub

Calls Zeebe: **starts** `blog-submission-workflow`

Subscribes to Redis channel `workflow:response:{correlationKey}` to **wait for response**

Step 6: Camunda Zeebe → `blog-submission-workflow.bpmn`

```
StartEvent → validate-entity-data → blog-postgres → email-send
(Ops) → send-notification (User) → send-api-response → End
```

`validate-entity-data` worker: Checks `entityType = "blog"`, formData present, etc.

`blog-postgres` worker: Receives job.

Enforces `status = 'PENDING_REVIEW'` regardless of what the frontend sent.

Saves blog into DB (`listings`, `blogs`, `listing_categories`, `listing_stats`, `author_profiles`).

Generates two Zeebe variables for the next email-send task:

`opsEmailHtml` — a clean HTML metadata table (see format below)

`opsAttachments` — a JSON array containing the full blog as a `blog-{id}.md` file, base64-encoded

`email-send` **(Ops Team Alert)**: Sends `opsEmailHtml` as the email body with `opsAttachments` as an attached `.md` file.

`send-notification` **(User Alert)**: Sends a "Blog Pending Review" confirmation email to the Author.

📧 Ops Team Email Format
Email Body (HTML Table — Metadata Only)
Ops team ko jo email dikhega uska body sirf metadata table hoga:

Field	Example Value
Blog ID	`550e8400-e29b-41d4-a716-446655440000`
Title	The Future of AI in 2025
Author	JaneTech
SEO Title	Future of AI 2025 | LeMiCi
Short Description	AI is changing the world...
Tags	AI, Tech
Reading Time	7 mins
Status	**PENDING_REVIEW**

> 📎 *"Full blog content is attached as* ***`blog-550e8400.md`****. Please review it and update the status on the admin dashboard."*

Email Attachment (`blog-{id}.md`)
Email ke saath ek Markdown file attach hogi jisme **poora blog content** hoga (koi truncation nahi):

```
# The Future of AI in 2025

**Author:** JaneTech
**Author Bio:** I write about AI and startups
**SEO Title:** Future of AI 2025 | LeMiCi
**SEO Description:** Explore how AI is reshaping every industry
in 2025
**Tags:** AI, Tech
```

```
**Reading Time:** 7 mins
**Status:** PENDING_REVIEW
**Blog ID:** 550e8400-e29b-41d4-a716-446655440000
**Featured Image:** https://s3.../blog-hero.jpg

---

## Content

<p>AI is transforming industries at an unprecedented pace...</p>
```

Technical Implementation

`blog-postgres` handler generates the **`.md`** content using **`strings.Builder`** and encodes it as **`base64`** using **`encoding/base64`**.

The variable **`opsAttachments`** is a JSON array in the format accepted by the **`email-send`** worker:

```
[
  {
    "filename": "blog-{id}.md",
    "contentType": "text/markdown",
    "content": "<base64-encoded-md-content>"
  }
]
```

The **`email-send`** worker parses this array and uses AWS SES **`SendRawEmail`** (MIME multipart) to attach the file.

No size limit concern: A typical blog post (5,000–10,000 words) is ~30–80 KB uncompressed, well within the AWS SES 10 MB limit.

Step 7: Ops Team Manual Review & Elasticsearch Sync

The Operations Team receives the HTML email with metadata and downloads the attached **`.md`** file to review the full content.

If approved, the Admin updates the blog status to **`LIVE`** in the database.

The Admin **manually** runs the **`sync-postgres-to-es.go`** script.

The script fetches all **`blog`** listings from Postgres and indexes them into Elasticsearch.

The blog is now searchable and visible on the **`/blog/listing`** page.
WHERE l.entity_type = 'blog'

```
→ Pushes flattened document to `blog_listings` Elasticsearch
index.

---

## 4. Complete API Reference

### Public Routes (No Auth Required)

| Method | Endpoint | Handler | Flow |
|--------|----------|---------|------|
| `GET` | `/api/v1/blog/home` | `GetHomeSections` | Zeebe →
`blog-home-page.bpmn` → `query-postgresql` (BLOG_FEATURED +
BLOG_POPULAR in **parallel**) → Postgres |
| `GET` | `/api/v1/blog/listing?
search=&category_id=&page=&page_size=` | `GetListing` | Zeebe →
`blog-listing-page.bpmn` → `query-postgresql` (BLOG_LISTING +
BLOG_POPULAR in parallel) → Postgres |
| `GET` | `/api/v1/blog/featured` | `GetFeatured` | **Direct**
Postgres query (no Zeebe) |
| `GET` | `/api/v1/blog/popular` | `GetPopular` | **Direct**
Postgres query (no Zeebe) |
| `GET` | `/api/v1/blog/:id` | `GetSingleBlog` | Zeebe → `blog-
detail-page.bpmn` → `query-postgresql` (BLOG_DETAIL) → Postgres.
**Also increments view_count async** |
| `POST` | `/api/v1/blog/subscribe` | `SubscribeNewsletter` |
Zeebe → `blog-submission-workflow` → `blog-postgres`
(`CREATE_SUBSCRIBER`) |

### Protected Routes (JWT Auth Required)

| Method | Endpoint | Handler | Flow |
|--------|----------|---------|------|
| `POST` | `/api/v1/blog` | `CreateBlog` | Zeebe → `blog-
submission-workflow` → `blog-postgres` (`CREATE_BLOG`) +
**Upserts author_profiles** |
| `PUT` | `/api/v1/blog/:id` | `UpdateBlog` | Zeebe → `blog-
submission-workflow` → `blog-postgres` (`UPDATE_BLOG`) |
| `POST` | `/api/v1/blog/:id/submit` | `UpdateBlog` | Same as
above (alias for status change like DRAFT→LIVE) |
| `GET` | `/api/v1/blog/media/presigned` | `GeneratePresignedURL`
```

```
| Direct → AWS S3 SDK → returns temp upload URL |
| `POST` | `/api/v1/authors/:author_id/follow` | `FollowAuthor` |
Zeebe → `blog-submission-workflow` → `blog-postgres`
(`CREATE_FOLLOWER`) |
| `DELETE` | `/api/v1/authors/:author_id/follow` |
`UnfollowAuthor` | Zeebe → `blog-submission-workflow` → `blog-
postgres` (`DELETE_FOLLOWER`) |

---

## 5. BPMN Workflows Used

| BPMN Process ID | Triggered By | What it Does |
|-----------------|-------------|--------------|
| `blog-submission-workflow` | All write operations | validate →
save to Postgres → notify |
| `blog-home-page` | `GET /blog/home` | Fetches Featured +
Popular **in parallel** via two concurrent `query-postgresql`
jobs |
| `blog-listing-page` | `GET /blog/listing` | Fetches Blog List +
Popular in parallel |
| `blog-detail-page` | `GET /blog/:id` | Fetches single blog full
detail + related articles |

---

## 6. Key Design Decisions

| Decision | Reason |
|----------|--------|
| `/featured` and `/popular` bypass Zeebe | They are lightweight
widgets. Starting a full Zeebe workflow for a simple SELECT query
adds 500ms+ latency unnecessarily. |
| BPMN not changed for Author Profiles | The BPMN blindly
forwards the entire JSON payload. We just added new fields to
that JSON. The existing worker reads what it needs. |
| Author profile upserted on blog create | Avoids a separate API
call. The blog creation form collects both profile info and blog
info together, so they're saved in one transaction. |
| view_count incremented in goroutine | `go db.Exec(...)` — Non-
blocking. The user gets the blog response instantly without
waiting for the counter update. |
```

| Redis PubSub for sync workflow response | API Gateway starts a Zeebe workflow and then waits on a Redis channel. The `send-api-response` worker publishes the result back after all BPMN steps complete. |